

Razvoj web aplikacija - Predavanje 3

Autentikacija i sigurnost (JWT)

mag.ing. Josip Torić (SIT UNIZD)

19.10.2026.



Plan predavanja (1/2)

- 1 Zašto autentikacija nije trivijalna (gdje greška boli)
- 2 Hashiranje lozinki: bcrypt i zašto ne MD5/SHA
- 3 JWT struktura: header, payload, signature
- 4 Access token i refresh token: uloge i životni ciklus
- 5 Login i refresh endpointi u FastAPI-ju

Plan predavanja (2/2)

- 6 Dependency `get_current_user`
- 7 Token storage: `localStorage` vs `HttpOnly` cookie
- 8 XSS i CSRF: što su i koji storage što štiti
- 9 Testovi za auth endpointe

Na kraju predavanja imate funkcionalan login flow: korisnik se može prijaviti, dobiti token i koristiti zaštićene endpointe.

Ishodi učenja

Nakon ovog predavanja student može:

- objasniti zašto lozinke hashiramo i što je salt
- opisati strukturu JWT tokena i što svaki dio radi
- objasniti razliku između access i refresh tokena
- implementirati `get_current_user` dependency u FastAPI-ju
- usporediti `localStorage` i `HttpOnly` cookie kao strategije pohrane
- objasniti XSS i CSRF napade i koji storage što štiti
- napisati test koji provjerava login i zaštićeni endpoint

Kontekst: gdje smo u projektu?

- Predavanje 2: imamo tablice **Club** i **User**, migracije i seed
- Seed je kreirao admin korisnika i demo klub korisnika
- Danas: ti korisnici se mogu **prijaviti** i dobiti token

Sljedeći korak (Predavanje 4): autorizacija - **tko smije što raditi** (role i ownership). Autentikacija je samo: **tko si ti**.

Zašto autentikacija nije trivijalna?

- Greška u auth = napadač dobiva pristup tuđim podacima
- Najčešće greške iz prakse:
 - lozinka pohranjena u plaintext ili slabo hashirana
 - JWT tajni ključ je „secret” ili prazan string
 - token se nikad ne invalidira (nema expiry)
 - osjetljivi podaci u JWT payloadu - payload je čitljiv svakome

Auth je jedan od rijetkih dijelova gdje „radi mi” nije dovoljno. Mora raditi i **sigurno**.

Lozinke: zašto ne smijemo pohranjivati plaintext?

- Ako napadač ukrade bazu: odmah ima pristup svim računima
- Isti problem vrijedi za slabe hasheve (MD5, SHA-1, SHA-256 bez salta)
 - rainbow tablice i brute force su trivijalni
- Rješenje: **spori hash s per-password saltom**

Baza se može ukrasti. Lozinke moraju biti beskorisne napadaču čak i tada.

bcrypt: kako radi (intuitivno)

- bcrypt generira **random salt** pri svakom hashiranju
- Salt se ugradi u hash string - nema potrebe za posebnim stupcem
- **Work factor** (cost): koliko je hash računski skup
 - viši cost = sporiji napad, ali i sporija provjera
 - preporučeni cost: 12 - dobar balans sigurnosti i brzine
- Ista lozinka → različit hash svaki put (salt je drugačiji)

Na obrani pitam: što je salt i zašto se zbog njega isti hash nikad ne ponavlja za istu lozinku?

bcrypt u Pythonu

```
import bcrypt

def hash_password(plain: str) -> str:
    return bcrypt.hashpw(
        plain.encode(), bcrypt.gensalt()
    ).decode()

def verify_password(plain: str, hashed: str) -> bool:
    return bcrypt.checkpw(
        plain.encode(), hashed.encode()
    )
```

Koristimo bcrypt direktno, ne passlib wrapper - passlib ima problem kompatibilnosti s `bcrypt ≥ 4.1`. `checkpw` interno radi constant-time usporedbu - to je zaštita od timing napada.

JWT: što je i što nije

- JWT = JSON Web Token - standardizirani format (RFC 7519)
- Token je **potpisan**, ali **nije enkriptiran** (osim JWE)
 - payload se može pročitati bez ključa (base64url decode)
 - stoga: ne stavljajte tajne podatke u payload
- Potpis garantira: **token nije modificiran i izdao ga je naš server**

Česta greška: misleći da je JWT „enkriptiran”, developer u payload stavlja lozinku ili osjetljive osobne podatke.

JWT struktura: header.payload.signature

Token izgleda ovako (tri dijela odvojena točkom):

xxxxxx.yyyyyy.zzzzzz

- **Header**: algoritam i tip tokena
 - { "alg": "HS256", "typ": "JWT" }
- **Payload**: claims (podaci o korisniku i tokenu)
 - sub (subject), exp (expiry), iat (issued at), custom claims
- **Signature**: HMAC-SHA256(base64(header) + "." + base64(payload), secret)

JWT Claims: što stavljamo u payload?

- **Standardni claims** (preporučeni):
 - sub: identifikator korisnika (npr. "42")
 - exp: Unix timestamp isteka tokena
 - iat: Unix timestamp izdavanja
 - iss: issuer (npr. "sit-unizd")
- **Custom claims** (naše aplikacije):
 - role: "admin" ili "club"
 - club_id: FK za ownership provjere

Payload mora biti **minimalan**. Samo ono što treba za autorizaciju - ništa više.

Access token vs Refresh token

	Access token	Refresh token
Trajanje	kratko (15 min)	dugo (7-30 dana)
Gdje se šalje	svaki API request	samo na /auth/refresh
Gdje se čuva	memory / localStorage	localStorage / HttpOnly cookie
Ako iscure	kratak prozor rizika	ozbiljan problem

Access token je **kratkotrajan** jer ga ne možemo invalidirati bez centralnog stanja. Refresh token omogućava „tiho” obnavljanje bez ponovne prijave.

Životni ciklus tokena (flow)

- 1 Korisnik šalje username + lozinku na POST /auth/login
- 2 Backend verificira lozinku, izdaje **access** i **refresh** token
- 3 Frontend pohranjuje tokene - o tome u nastavku
- 4 Na svakom zaštićenom requestu: šalje access token u Authorization: Bearer ...
- 5 Kad access token istekne (401): frontend tiho poziva POST /auth/refresh
- 6 Backend verificira refresh token, izdaje novi access token
- 7 Korisnik ne primjećuje ništa

JWT utils: izdavanje i verifikacija

```
from datetime import datetime, timedelta, timezone
import jwt

def create_access_token(user_id: int, role: str,
                        club_id: int | None = None) -> str:
    payload = {
        "sub": str(user_id),
        "role": role,
        "club_id": club_id,
        "exp": datetime.now(timezone.utc) + timedelta(minutes=15),
        "iat": datetime.now(timezone.utc),
        "iss": settings.JWT_ISSUER,
    }
    return jwt.encode(payload, settings.JWT_SECRET, algorithm="HS256")

def decode_token(token: str) -> dict:
    try:
        return jwt.decode(token, settings.JWT_SECRET,
                           algorithms=["HS256"],
                           issuer=settings.JWT_ISSUER,
                           options={"require": ["exp"]})
    except jwt.ExpiredSignatureError:
        raise UnauthorizedError("Token expired")
    except jwt.InvalidTokenError:
        raise UnauthorizedError("Invalid token")
```

Koristimo **PyJWT** - aktivno održavana biblioteka. Razlikovanje `ExpiredSignatureError` od `InvalidTokenError` omogućuje da frontend zna treba li refresh (istekao) ili ponovni login (nevažeći).

Login endpoint

```
from fastapi import APIRouter, Depends
from sqlalchemy.ext.asyncio import AsyncSession
from app.core.deps import get_db
from app.schemas.auth import LoginRequest, TokenResponse
from app.services.auth import authenticate_user, create_tokens

router = APIRouter()

@router.post("/login", response_model=TokenResponse)
async def login(body: LoginRequest,
               db: AsyncSession = Depends(get_db)):
    user = await authenticate_user(db, body.username, body.password)
    access_token, refresh_token = create_tokens(user)
    return TokenResponse(access_token=access_token,
                        refresh_token=refresh_token)
```

`authenticate_user` je u service layeru, ne u routeru.

Refresh endpoint

```
@router.post("/refresh", response_model=TokenResponse)
async def refresh(body: RefreshRequest,
                  db: AsyncSession = Depends(get_db)):
    payload = decode_token(body.refresh_token)
    user = await get_user_by_username(db, payload["sub"])
    if not user or not user.is_active:
        raise UnauthorizedError("User not found or inactive")
    access_token, new_refresh = create_tokens(user)
    return TokenResponse(access_token=access_token,
                        refresh_token=new_refresh)
```

Razmislite do obrane: zašto provjeravamo `is_active`, a ne samo potpis tokena?

Dependency: get_current_user

```
from fastapi import Depends, Header
from app.core.errors import UnauthorizedError

async def get_current_user(
    authorization: str = Header(...),
    db: AsyncSession = Depends(get_db)
) -> User:
    if not authorization.startswith("Bearer "):
        raise UnauthorizedError("Missing token")
    token = authorization.removeprefix("Bearer ")
    payload = decode_token(token)
    user = await get_user_by_id(db, payload["sub"])
    if not user or not user.is_active:
        raise UnauthorizedError("User not found")
    return user
```

Korištenje dependencyja u routeru

```
@router.get("/me")
async def get_me(current_user: User = Depends(get_current_user)):
    return {"id": current_user.id, "username": current_user.username,
            "role": current_user.role}

@router.get("/clubs")
async def list_clubs(
    db: AsyncSession = Depends(get_db),
    current_user: User = Depends(get_current_user)
):
    # svaki korisnik koji je prijavljen smije vidjeti klubove
    return await get_all_clubs(db)
```

Provjera **role i ownership** dolazi u Predavanju 4.

Zašto dependency, a ne middleware za auth?

- Middleware se izvršava za **sve** requestove (i `/health`, i `/docs`)
- Dependency se izvršava samo gdje ga **deklariramo**
- Dependency je **testabilniji**: lako ga možemo zamijeniti u testovima
- FastAPI dependency injection je eksplicitan:
 - odmah vidimo koji endpoint treba autentikaciju

Pitanje za razmišljanje: kako bi testirao endpoint koji koristi `get_current_user`?
Odgovor stiže u Predavanju 11.

Token storage: gdje čuvamo tokene na frontendu?

Dvije uobičajene opcije:

	localStorage	HttpOnly cookie
Pristup JS-u	da	ne
XSS ranjivost	visoka	niska
CSRF ranjivost	niska	umjerena
Implementacija	jednostavna	složenija
Pogodnost za SPA	visoka	zahtijeva credentials

U projektu koristimo **localStorage** za oba tokena - jednostavnija implementacija za SPA. HttpOnly cookie je sigurnija produkcijska alternativa, pokazat ćemo i nju.

XSS (Cross-Site Scripting): što je i zašto boli

- Napadač ubaci zloćudni JavaScript u stranicu (npr. kroz komentar, URL)
- Taj JS se izvršava u browseru žrtve - ima isti pristup kao vaš kod
- Ako je token u localStorage:
 - `document.cookie` nema token, ali `localStorage.getItem()` ga ima
 - napadač krade token s jednom linijom koda
- Zaštita: HttpOnly cookie (JS ga ne može pročitati), CSP header, sanitizacija outputa

CSRF (Cross-Site Request Forgery): što je i zašto boli

- Napadač nagovori korisnika da posjeti stranicu koja šalje request u **korisnikovo ime**
- Browser automatski šalje cookieje uz request (uključujući auth cookie)
- Ako koristite HttpOnly cookie bez CSRF zaštite: ranjivi ste na CSRF
- Zaštita: SameSite=Strict/Lax na cookie, CSRF token, provjera Origin headera

localStorage + XSS = krađa tokena.

HttpOnly cookie + CSRF = izvršavanje akcija.

Nema savršene opcije - bira se manji rizik.

Praktična odluka za naš projekt

- **Access i refresh token:** `localStorage`
 - preživljava refresh stranice - korisnik ostaje prijavljen
 - frontend implementaciju radimo u Predavanju 8
- Cijena: token je dostupan JavaScriptu, XSS ga može ukrasti
- Zato: kratak access token (15 min), Vue auto-escaping, nikad `v-html` s korisničkim sadržajem

U produkcijskim sustavima refresh token ide u **HttpOnly cookie** - JS ga ne može pročitati. Za naš projekt `localStorage` je svjesni trade-off, ne neznanje.

Na obrani pitam: što napadač dobiva XSS-om ako su tokeni u `localStorageu`?
Kako **HttpOnly cookie** mijenja rizik?

Produksijska alternativa: HttpOnly cookie (FastAPI)

```
from fastapi import Response

@router.post("/login")
async def login(body: LoginRequest, response: Response,
               db: AsyncSession = Depends(get_db)):
    user = await authenticate_user(db, body.username, body.password)
    access_token, refresh_token = create_tokens(user)
    response.set_cookie(
        key="refresh_token",
        value=refresh_token,
        httponly=True,
        secure=True, # samo HTTPS u produkciji
        samesite="lax",
        max_age=60 * 60 * 24 * 30, # 30 dana
    )
    return {"access_token": access_token}
```

Testovi za auth: što pokrivamo?

- **Uspješan login:** točni kredencijali → 200 + token
- **Krivi podaci:** pogrešna lozinka → 401
- **Nepostojeći username:** → 401 (isti error, ne otkrivamo postoji li)
- **Zaštićeni endpoint bez tokena:** → 401
- **Zaštićeni endpoint s tokenom:** → 200
- **Istekli token:** → 401
- **Refresh:** valjan refresh token → novi access token

Zašto isti error za krivu lozinku i nepostojeći username? Enumeracija korisnika: napadač bi mogao otkriti koji usernamei postoje u sustavu.

Primjer testa (pytest + TestClient)

```
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_login_success(db_with_seed):
    r = client.post("/auth/login",
                    json={"username": "admin",
                          "password": "admin123"})
    assert r.status_code == 200
    assert "access_token" in r.json()

def test_login_wrong_password(db_with_seed):
    r = client.post("/auth/login",
                    json={"username": "admin",
                          "password": "wrong"})
    assert r.status_code == 401

def test_protected_without_token():
    r = client.get("/auth/me")
    assert r.status_code == 401
```

Override dependencyja u testovima

```
from app.core.deps import get_current_user
from app.models.user import User

def fake_admin():
    return User(id=1, username="admin",
                role="admin", is_active=True)

app.dependency_overrides[get_current_user] = fake_admin

def test_me_endpoint():
    r = client.get("/auth/me")
    assert r.status_code == 200
    assert r.json()["role"] == "admin"

# uvijek čoistite override nakon testa
app.dependency_overrides = {}
```

Testiramo endpoint **neovisno** o auth logici.

Sigurnost JWT tajnog ključa

- Tajni ključ **mora biti** nasumičan i dugačak (min. 32 bajta)
 - `openssl rand -hex 32`
- Nikad ne smije biti u kodu ili u repozitoriju
- Env var: `JWT_SECRET` iz `.env` (koji je u `.gitignore`)
- Ako se kompromitira: **svi tokeni su nevaljani** - morate rotirati ključ

Zadatak za znatiželjne: kako bi invalidirao sve aktivne tokene odjednom, a da ne mijenjaš `JWT_SECRET`?

Brute force na login: rate limiting

Login endpoint je meta za pogađanje lozinki. bcrypt usporava svaki pojedinačni pokušaj, ali ne sprječava napadača da ih pošalje tisuće.

Mjera	Što sprječava
bcrypt s visokim custom rate limiting	offline crackanje ukradenih hasheva online pogađanje kroz login endpoint

Rate limiting ograničava broj pokušaja po IP-u ili usernameu, npr. 5 pokušaja u minuti. U FastAPI ekosustavu to radi biblioteka `slowapi`. Za studentski projekt nije obavezan, ali na obrani morate znati objasniti zašto bi ga produkcijski sustav imao.

Token invalidacija: ograničenja stateless JWT-a

- JWT je **stateless**: server ne prati koji su tokeni aktivni
- Ako se korisnik odjavi ili promijeni lozinku, stari token i dalje vrijedi do expiry-ja
- Opcije za invalidaciju:
 - **Kratki expiry** (access token): prozor rizika je mali
 - **Revocation list** (blacklist): centralna lista invalidnih tokena (Redis)
 - **Token version** u bazi: ako se ne poklapa, token je nevaljan

Za ovaj projekt: kratki access token + refresh token je dovoljan. Revocation list je „nice to have”.

Commitovi za Predavanje 3

- `feat(api): add bcrypt helpers, auth schemas and user repository`
- `feat(api): add JWT utilities (create/decode access + refresh tokens)`
- `feat(api): add auth service and login/refresh endpoints`
- `feat(api): add get_current_user dependency and /auth/me endpoint`
- `test(api): add auth tests (login, refresh, expired token, protected endpoint)`

Na satu: prolazimo diffeve i testiramo sa Swaggerom - login, token, zaštićeni endpoint.

Što ide sljedeće (Predavanje 4)

- Autorizacija: tko smije što raditi
- Role check: admin vs club korisnik
- Ownership: korisnik smije mijenjati samo svoje
- Dependency `require_role` i `require_admin`
- Greške: 401 vs 403 - razlika i kad što vraćamo